

DE .ONNX A .HEF EN HAILO-8

Tutorial definitivo — probado empíricamente en Ubuntu

Raspberry Pi 5 + Hailo-8 AI Hat | YOLOv8n-Pose | 12 keypoints

Centro	IES Politecnico Jesus Marin, Malaga
Proyecto	Deteccion orientacion giroscopio drones (Roll/Pitch/Yaw)
Hardware objetivo	Raspberry Pi 5 + Hailo-8 AI Hat
Imagen Docker	hailo8_ai_sw_suite_2025-10:1
Hailo DFC / HailoRT	v3.33.0 / v4.23.0
Modelo	giroscopio_pose_best_n_g1.onnx (YOLOv8n-Pose, 640x640, 12 KP)
Workspace local	/home/nutshell/Documentos/hailo_workspace
Workspace en Docker	/workspace (montado con -v, solo lectura para hailo)
Directorio de trabajo	~ (/home/hailo) — SIEMPRE ejecutar comandos desde aqui

★ **Estado de validacion**

El flujo Ubuntu esta completamente probado y verificado en sesion real (abril 2026). El flujo Windows es equivalente teorico — el proceso dentro del contenedor Docker es identico, solo cambia la forma de instalar Docker y montar el volumen.

Indice

PARTE 1 — UBUNTU (probado)

- 1.1 [Instalar Docker en Ubuntu](#)
- 1.2 [Descargar e instalar la imagen Hailo Docker](#)
- 1.3 [Preparar el workspace](#)
- 1.4 [Arrancar el contenedor](#)
- 1.5 [Convertir imagenes de calibracion a .npy](#)
- 1.6 [Parsear: .onnx → .har](#)
- 1.7 [Optimizar y cuantizar: .har → .har optimizado](#)
- 1.8 [Compilar: .har → .hef](#)
- 1.9 [Guardar archivos y salir del contenedor](#)

1.10 Copiar .hef a la Raspberry Pi 5

PARTE 2 — WINDOWS (equivalente teorico)

- 2.1 Instalar Docker Desktop en Windows
- 2.2 Descargar e instalar la imagen Hailo Docker
- 2.3 Preparar el workspace
- 2.4 Arrancar el contenedor
- 2.5 Pasos 5-10 identicos al flujo Ubuntu

PARTE 3 — REFERENCIA

- 3.1 Chuleta: secuencia completa Ubuntu de una vez
- 3.2 Tabla de errores reales y soluciones
- 3.3 Archivos generados y para que sirve cada uno

PARTE 1 — UBUNTU (probado empíricamente)

1.1 Instalar Docker en Ubuntu

Si Docker no esta instalado en tu maquina Ubuntu, ejecuta:

```
# Actualizar paquetes
sudo apt update && sudo apt upgrade -y

# Instalar dependencias
sudo apt install -y ca-certificates curl gnupg lsb-release

# Anadir clave GPG oficial de Docker
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg
--dearmor -o /etc/apt/keyrings/docker.gpg

# Anadir repositorio Docker
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Instalar Docker Engine
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io

# Anadir tu usuario al grupo docker (para no necesitar sudo)
sudo usermod -aG docker $USER

# Verificar instalacion (reinicia sesion antes si es la primera vez)
docker --version
```

! Reinicio de sesion necesario

Despues de hacer usermod, cierra sesion y vuelve a entrar para que el grupo docker surta efecto. Sin esto necesitaras poner sudo delante de cada comando docker.

1.2 Descargar e instalar la imagen Hailo Docker

La imagen Docker con el compilador Hailo (Dataflow Compiler) no esta en Docker Hub. Hay que descargarla del portal oficial de desarrolladores de Hailo.

- Ve a: <https://hailo.ai/developer-zone/>
- Inicia sesion (requiere registro gratuito)
- Busca la seccion de descargas de Software Suite
- Descarga la version que corresponda al firmware de tu chip Hailo-8
- El archivo descargado es un .tar.gz (en nuestro caso: hailo8_ai_sw_suite_2025-10.tar.gz)

Una vez descargado, carga la imagen en Docker:

```
# Cargar la imagen (puede tardar varios minutos, pesa ~17GB)
docker load < hailo8_ai_sw_suite_2025-10.tar.gz

# Verificar que se cargo correctamente
docker images
```

Debes ver algo como:

REPOSITORY	TAG	IMAGE ID	SIZE
hailo8_ai_sw_suite_2025-10	1	3a94fa154926	17.5GB

! El tag es :1, no :latest

Docker asume el tag :latest si no especificas ninguno. Como la imagen tiene tag :1, siempre debes incluirlo explicitamente (hailo8_ai_sw_suite_2025-10:1). Sin el tag, Docker busca :latest en internet y falla con 'pull access denied'.

1.3 Preparar el workspace

Crea la estructura de carpetas necesaria en Ubuntu:

```
mkdir -p /home/nutshell/Documentos/hailo_workspace/calib
```

Copia tu modelo .onnx a hailo_workspace/ y las imagenes de calibracion a hailo_workspace/calib/. La estructura debe quedar asi:

```
hailo_workspace/  
  giroscopio_pose_best_n_g1.onnx  <-- modelo entrenado  
  calib/                          <-- 100-300 imagenes JPG/PNG  
    imagen_001.jpg  
    imagen_002.jpg  
    ...
```

i Sobre las imagenes de calibracion

Deben ser imagenes representativas del uso real: el giroscopio en distintas orientaciones, iluminaciones y fondos. Con 100-300 imagenes es suficiente. NO es necesario que sean exactamente 640x640, el script de conversion las redimensiona automaticamente.

1.4 Arrancar el contenedor Docker

Ejecuta este comando desde cualquier terminal en Ubuntu:

```
docker run -it --rm \  
  -v /home/nutshell/Documentos/hailo_workspace:/workspace \  
  hailo8_ai_sw_suite_2025-10:1 bash
```

-it	Terminal interactiva dentro del contenedor
--rm	Elimina el contenedor al salir (no deja rastros). ATENCION: los archivos generados en ~ se pierden al hacer exit.
-v ruta:dest	Monta hailo_workspace como /workspace dentro del contenedor. Los archivos que copies a /workspace desde ~ seran visibles en Ubuntu.

El prompt cambia a algo como:

```
(hailo_virtualenv) hailo@d07f0c1431f6:/local/workspace$
```

Lo primero que debes hacer siempre al entrar es ir a ~:

```
cd ~
```

```
ls /workspace # debe mostrar tu .onnx y la carpeta calib/
```

! REGLA DE ORO: trabaja siempre desde ~

/workspace es de solo lectura para el usuario hailo. Si intentas ejecutar cualquier comando hailo desde /workspace obtendras: Permission denied: /workspace/pyhailort.log. Ejecuta SIEMPRE desde ~ y apunta a /workspace solo para leer archivos.

1.5 Convertir imagenes de calibracion a .npy

El optimizador Hailo NO acepta JPG/PNG directamente. Necesita arrays NumPy (.npy). Este paso es obligatorio y solo hay que hacerlo una vez por sesion (se pierde al salir del contenedor).

```
python3 << 'EOF'
import os, numpy as np
from PIL import Image

calib_dir = '/workspace/calib/'
out_dir = os.path.expanduser('~/.calib_npy/')
os.makedirs(out_dir, exist_ok=True)

files = [f for f in os.listdir(calib_dir) if
f.lower().endswith(('.jpg', '.png'))]
print(f'Convirtiendo {len(files)} imagenes...')

for i, fname in enumerate(files):
    img = Image.open(os.path.join(calib_dir, fname)).convert('RGB')
    img = img.resize((640, 640)) # ajustar al tamaño de entrada del
    modelo
    np.save(os.path.join(out_dir, f'calib_{i:04d}.npy'), np.array(img,
dtype=np.float32))

print(f'Listo: {len(files)} archivos .npy en {out_dir}')
EOF
```

Salida esperada:

```
Convirtiendo 299 imagenes...
Listo: 299 archivos .npy en /home/hailo/calib_npy/
```

! Error Stopliteration si se omite este paso

Si pasas `--calib-set-path` apuntando a una carpeta de JPGs directamente obtendras: `StopIteration` en `npz_dir_to_dataset`. El error con `.npy` incorrectos es: `Couldn't detect CalibrationDataType`. Ambos indican que las imagenes no estan en el formato correcto.

1.6 Parsear: `.onnx` → `.har`

Este paso traduce el modelo ONNX al formato intermedio de Hailo. Es critico especificar los 9 nodos finales correctos (las capas Conv de la cabeza de deteccion). Sin ellos el compilador falla con 'No valid partition found'.

```
# Asegurate de estar en ~
cd ~

hailo parser onnx /workspace/giroscopio_pose_best_n_g1.onnx \
--net-name giroscopio \
--end-node-names \
/model.22/cv2.0/cv2.0.2/Conv \
/model.22/cv3.0/cv3.0.2/Conv \
/model.22/cv4.0/cv4.0.2/Conv \
/model.22/cv2.1/cv2.1.2/Conv \
/model.22/cv3.1/cv3.1.2/Conv \
/model.22/cv4.1/cv4.1.2/Conv \
/model.22/cv2.2/cv2.2.2/Conv \
/model.22/cv3.2/cv3.2.2/Conv \
/model.22/cv4.2/cv4.2.2/Conv
```

Salida esperada:

```
[info] End nodes mapped from original model:
'/model.22/cv2.0/cv2.0.2/Conv', ...
[info] Translation completed on ONNX model giroscopio
[info] Saved HAR to: /home/hailo/giroscopio.har
```

Verificar que el archivo se genero:

```
ls -lh ~/giroscopio.har # debe mostrar ~13MB
```

! Por que estos 9 nodos y no otros

El parser automatico elige nodos Concat/Mul/Sigmoid como salida, lo que hace que el modelo no quepa en el chip (error: No valid partition found). Los nodos cv2/cv3/cv4 Conv son los que usa el Hailo Model Zoo para YOLOv8-Pose, y permiten que el compilador divida el modelo en dos contextos que si caben en el Hailo-8.

1.7 Optimizar y cuantizar: .har → .har optimizado

Este paso cuantiza el modelo de FP32 a INT8 usando las imagenes de calibracion y aplica fine-tuning para recuperar precision. Es el paso mas lento (~13 minutos en un i7-9700).

Primero crea el script .alls con los parametros de optimizacion:

```
cat > ~/giroscopio.alls << 'EOF'
normalization1 = normalization([0.0, 0.0, 0.0], [255.0, 255.0, 255.0])
change_output_activation(conv71, sigmoid)
change_output_activation(conv58, sigmoid)
change_output_activation(conv44, sigmoid)
pre_quantization_optimization(equalization, policy=disabled)
quantization_param(output_layer3, precision_mode=a16_w16)
quantization_param(output_layer6, precision_mode=a16_w16)
quantization_param(output_layer9, precision_mode=a16_w16)
post_quantization_optimization(finetune, policy=enabled,
learning_rate=0.00015)
EOF
```

Luego lanza el optimize:

```
hailo optimize ~/giroscopio.har \
--hw-arch hailo8 \
--calib-set-path ~/calib_npy/ \
--model-script ~/giroscopio.alls
```

Veras barras de progreso. Al terminar (~13 min):

```
Calibration: 100%|██████████| 64/64 [00:27<00:00]
[info] Model Optimization Algorithm Quantization-Aware Fine-Tuning is
done (completion time is 00:13:21)
[info] Model Optimization is done
[info] Saved HAR to: /home/hailo/giroscopio_optimized.har
```

i Que hace el script .alls

Normaliza la entrada (divide por 255), fuerza activacion sigmoid en las capas de confianza de deteccion, mantiene 16 bits en las capas de salida de keypoints (a16_w16) para mayor precision, y activa fine-tuning post-cuantizacion. Es el mismo script que usa el Hailo Model Zoo para YOLOv8-Pose.

1.8 Compilar: .har optimizado → .hef

El ultimo paso genera el binario final para el chip Hailo-8. Tarda ~8 minutos.

```
hailo compiler ~/giroscopio_optimized.har --hw-arch hailo8
```

Al terminar veras la utilizacion de cada cluster del chip y la confirmacion:

```
[info] Successful Mapping (allocation time: 8m 6s)
[info] Compiling kernels of giroscopio_context_0...
[info] Compiling kernels of giroscopio_context_1...
[info] Successful Compilation (compilation time: 8s)
[info] Saved HEF to: /home/hailo/giroscopio.hef
[info] Saved HAR to: /home/hailo/giroscopio_compiled.har
```

i El modelo usa 2 contextos

El Hailo-8 tiene que partir el modelo en dos contextos (context_0 y context_1) porque no cabe en uno solo. Esto es normal para YOLOv8-Pose. La utilizacion total es ~60% control, ~45% compute, ~40% memoria, dejando margen suficiente.

1.9 Guardar archivos y salir del contenedor

Con --rm los archivos de ~ se pierden al hacer exit. Antes de salir, copia todo a /workspace:

```
# Verificar que los archivos existen y tienen tamaño razonable
ls -lh ~/giroscopio.hef ~/giroscopio_optimized.har
~/giroscopio_compiled.har

# Copiar a /workspace (visible en Ubuntu como hailo_workspace/)
cp ~/giroscopio.hef /workspace/
cp ~/giroscopio_optimized.har /workspace/
cp ~/giroscopio_compiled.har /workspace/
cp ~/giroscopio.alls /workspace/

# Verificar que llegaron
ls -lh /workspace/*.hef /workspace/*.har /workspace/*.alls

# Solo cuando veas todos los archivos en /workspace, salir
exit
```

Tamano esperados:

```
-rw-rw-rw- 1 hailo ht 7.1M giroscopio.hef <-- ARCHIVO FINAL
```



```
-rw-rw-rw- 1 hailo ht   62M  giroscopio_optimized.har  <-- modelo
cuantizado
-rw-rw-rw- 1 hailo ht   69M  giroscopio_compiled.har   <-- modelo
compilado
-rw-rw-rw- 1 hailo ht    512  giroscopio.alls           <-- script
optimizacion
```

! No hagas exit hasta ver los archivos en /workspace

El comando exit con --rm es irreversible. Una vez ejecutado, todo lo que este en ~ desaparece para siempre. Verifica siempre con ls /workspace/*.hef antes de salir.

1.10 Copiar el .hef a la Raspberry Pi 5

Una vez fuera del contenedor, el .hef esta en hailo_workspace/ en Ubuntu. Copialo a la Raspberry Pi 5 por red:

```
# Sustituye <IP-RASPBERRY> por la IP real de tu RPi5
scp /home/nutshell/Documentos/hailo_workspace/giroscopio.hef \
pi@<IP-RASPBERRY>:~/
# Alternativa: pendrive USB si no tienes conexion de red
```

PARTE 2 — WINDOWS (equivalente teórico, no probado)

i Nota sobre esta seccion

El flujo de trabajo dentro del contenedor Docker es identico en Windows y Ubuntu. Las diferencias estan unicamente en como se instala Docker y como se especifica la ruta del volumen al lanzar el contenedor. Todo lo descrito en los pasos 1.5 a 1.10 aplica sin cambios.

2.1 Instalar Docker Desktop en Windows

- Descarga Docker Desktop desde: <https://www.docker.com/products/docker-desktop/>
- Ejecuta el instalador y sigue el asistente
- Durante la instalacion, asegurate de activar la opcion 'Use WSL 2 instead of Hyper-V' (recomendado)
- Reinicia el ordenador si se solicita
- Abre Docker Desktop y espera a que el motor arranque (icono de ballena en la barra de tareas)

i Requisito WSL2

Para el mejor rendimiento y compatibilidad con imagenes Linux como la de Hailo, es recomendable tener WSL2 instalado. Puedes instalarlo con: `wsl --install` en PowerShell como administrador.

2.2 Descargar e instalar la imagen Hailo Docker en Windows

El proceso de descarga es identico al de Ubuntu (portal hailo.ai/developer-zone/). Una vez descargado el .tar.gz, cargalo desde PowerShell o CMD:

```
# En PowerShell o CMD (no en WSL)
docker load -i hailo8_ai_sw_suite_2025-10.tar.gz

# Verificar
docker images
```

i Diferencia con Ubuntu

En Ubuntu se usa el operador `<` para redirigir. En Windows PowerShell se usa `-i`. El resultado es identico.

2.3 Preparar el workspace en Windows

Crea una carpeta en Windows, por ejemplo:

```
C:\Users\TuUsuario\hailo_workspace\calib\
```

Copia tu .onnx a hailo_workspace\ y las imagenes de calibracion a hailo_workspace\calib\.

2.4 Arrancar el contenedor Docker en Windows

La diferencia principal respecto a Ubuntu esta en la ruta del volumen. En PowerShell:

```
# En PowerShell
docker run -it --rm `
-v C:/Users/TuUsuario/hailo_workspace:/workspace `
hailo8_ai_sw_suite_2025-10:1 bash
```

En CMD:

```
docker run -it --rm ^
-v C:/Users/TuUsuario/hailo_workspace:/workspace ^
hailo8_ai_sw_suite_2025-10:1 bash
```

i Diferencias de sintaxis Windows vs Ubuntu

En PowerShell el caracter de continuacion de linea es el acento grave (`) en lugar de la barra invertida (\). En CMD es el circunflejo (^). Las rutas de Windows usan / en lugar de \ cuando se pasan a Docker. Todo lo demas es identico.

Una vez dentro del contenedor, los pasos 1.5 a 1.10 son exactamente iguales que en Ubuntu. El entorno dentro del contenedor es Linux independientemente del sistema operativo del host.

PARTE 3 — REFERENCIA RAPIDA

3.1 Chuleta: secuencia completa Ubuntu de una vez

Copia y pega esta secuencia completa. El unico paso interactivo es el parser (puede preguntar si reintentar — responder y). NO incluye exit al final para evitar salir antes de verificar.

```
# — ARRANCAR DOCKER —————
docker run -it --rm \
  -v /home/nutshell/Documentos/hailo_workspace:/workspace \
  hailo8_ai_sw_suite_2025-10:1 bash

# — DENTRO DEL CONTENEDOR —————
cd ~

# PASO 1: Convertir imagenes de calibracion a .np
python3 << 'PYEOF'
import os, numpy as np
from PIL import Image
calib_dir='/workspace/calib/'; out_dir=os.path.expanduser('~'/calib_npy/')
os.makedirs(out_dir, exist_ok=True)
files=[f for f in os.listdir(calib_dir) if
f.lower().endswith(('.jpg','.png'))]
for i,f in enumerate(files):

img=Image.open(os.path.join(calib_dir,f)).convert('RGB').resize((640,640)
)

np.save(os.path.join(out_dir,f'calib_{i:04d}.npy'),np.array(img,dtype=np.
float32))
print(f'OK: {len(files)} archivos .npy')
PYEOF

# PASO 2: Parsear
hailo parser onnx /workspace/giroscopio_pose_best_n_g1.onnx \
  --net-name giroscopio \
  --end-node-names \
    /model.22/cv2.0/cv2.0.2/Conv \
    /model.22/cv3.0/cv3.0.2/Conv \
    /model.22/cv4.0/cv4.0.2/Conv \
```

```
/model.22/cv2.1/cv2.1.2/Conv \  
/model.22/cv3.1/cv3.1.2/Conv \  
/model.22/cv4.1/cv4.1.2/Conv \  
/model.22/cv2.2/cv2.2.2/Conv \  
/model.22/cv3.2/cv3.2.2/Conv \  
/model.22/cv4.2/cv4.2.2/Conv
```

```
# PASO 3: Crear script .alls
```

```
cat > ~/giroscopio.alls << 'EOF'  
normalization1 = normalization([0.0, 0.0, 0.0], [255.0, 255.0, 255.0])  
change_output_activation(conv71, sigmoid)  
change_output_activation(conv58, sigmoid)  
change_output_activation(conv44, sigmoid)  
pre_quantization_optimization(equalization, policy=disabled)  
quantization_param(output_layer3, precision_mode=a16_w16)  
quantization_param(output_layer6, precision_mode=a16_w16)  
quantization_param(output_layer9, precision_mode=a16_w16)  
post_quantization_optimization(finetune, policy=enabled,  
learning_rate=0.00015)  
EOF
```

```
# PASO 4: Optimizar (~13 min)
```

```
hailo optimize ~/giroscopio.har \  
--hw-arch hailo8 \  
--calib-set-path ~/calib_npy/ \  
--model-script ~/giroscopio.alls
```

```
# PASO 5: Compilar (~8 min)
```

```
hailo compiler ~/giroscopio_optimized.har --hw-arch hailo8
```

```
# PASO 6: Verificar y copiar (NO hacer exit hasta ver los archivos en  
/workspace)
```

```
ls -lh ~/giroscopio.hef ~/giroscopio_optimized.har  
~/giroscopio_compiled.har  
cp ~/giroscopio.hef /workspace/  
cp ~/giroscopio_optimized.har /workspace/  
cp ~/giroscopio_compiled.har /workspace/  
cp ~/giroscopio.alls /workspace/  
ls -lh /workspace/*.hef /workspace/*.har
```

```
# — SOLO CUANDO VEAS LOS ARCHIVOS EN /workspace
exit

# — DE VUELTA EN UBUNTU: copiar a Raspberry Pi 5
scp /home/nutshell/Documentos/hailo_workspace/giroscopio.hef pi@<IP>:~/
```

3.2 Tabla de errores reales y soluciones

Error	Causa	Solucion
pull access denied	Tag :latest no existe en local	Usar siempre hailo8_ai_sw_suite_2025-10:1 (con el tag :1)
Permission denied pyhailort.log	/workspace es read-only	Ejecutar SIEMPRE desde ~, nunca desde /workspace
StopIteration numpy_dir_to_dataset	calib/ contenia JPGs, no .numpy	Convertir con el script Python antes del optimize
Couldn't detect CalibrationDataType	carpeta .numpy vacia o mal generada	Verificar con ls ~/calib_numpy/ que hay archivos .numpy
--calib-path not recognized	Parametro incorrecto	Usar --calib-set-path (no --calib-path)
Model requires quantized weights	Se compilo el .har sin optimizar	Ejecutar hailo optimize antes de hailo compiler
No valid partition found	Nodos finales del parser incorrectos	Usar los 9 nodos cv2/cv3/cv4 Conv con --end-node-names
FileNotFoundError giroscopio.har	Parser ejecutado desde directorio incorrecto	Hacer cd ~ antes de ejecutar el parser
exit prematuro, archivos perdidos	--rm elimina todo al salir	Verificar ls /workspace/*.hef antes de hacer exit

3.3 Archivos generados y para que sirve cada uno

Archivo	Tamano	Descripcion
giroscopio.har	~13 MB	Modelo parseado (FP32). Salida del parser.
giroscopio.all	~512 B	Script de optimizacion. Guardar para reutilizar.
calib_numpy/	variable	Imagenes de calibracion en formato .numpy. Se regenera en cada sesion.
giroscopio_optimized.har	~62 MB	Modelo cuantizado INT8 con fine-tuning. Guardar para no repetir el optimize.
giroscopio_compiled.har	~69 MB	Modelo compilado con mapeo al chip. Util para depuracion.
giroscopio.hef	7.1 MB	ARCHIVO FINAL. El unico que se carga en la Raspberry Pi 5.